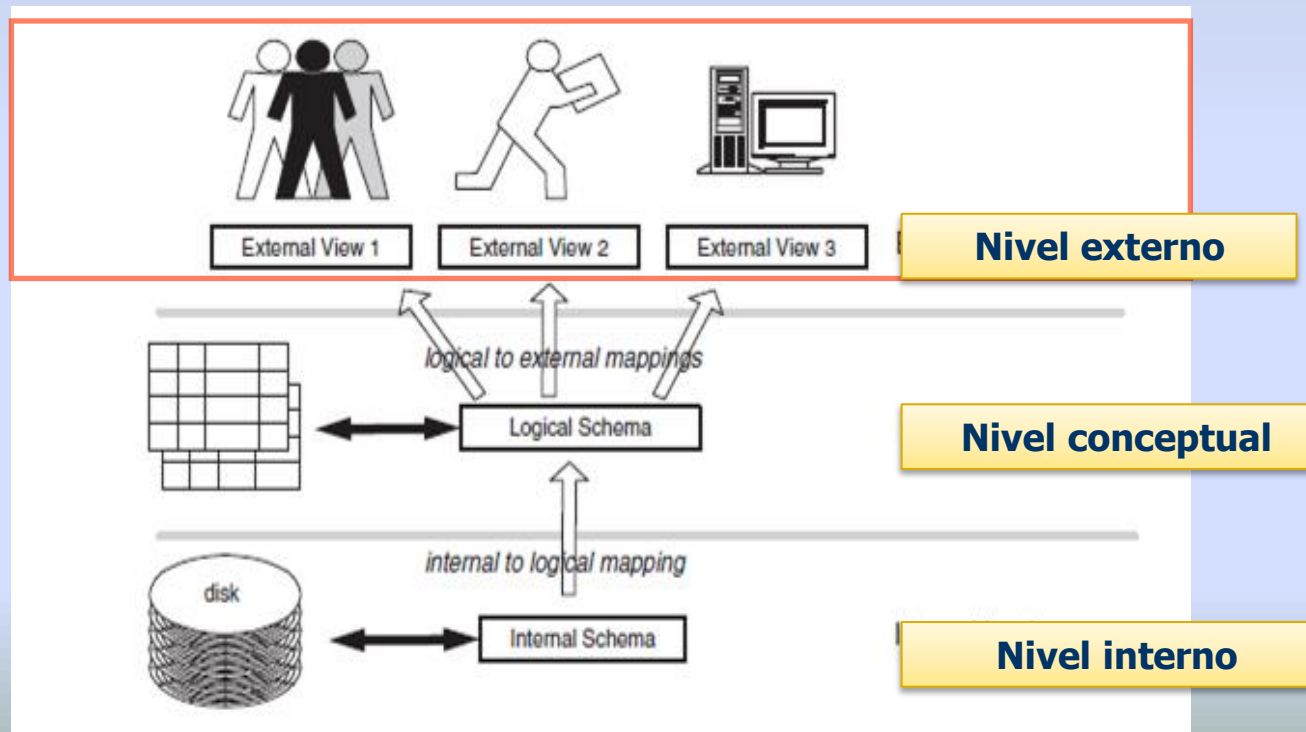




Vistas – Esquema Externo

- Forman parte del **esquema externo** de la base de datos
- **Presentan** una parte de la BD que es de interés para grupos particulares de usuarios (**ocultando** el resto de la información)



Concepto de Vista

- Es una **relación derivada** de una o más tablas y/o vistas definidas previamente
- Su contenido se define dando un nombre a una expresión de consulta
- Se considera una tabla **virtual** (habitualmente no materializada)
 - las tuplas se generan al operar sobre la vista
 - Pueden consultarse como cualquier tabla
 - Para poder realizar **actualizaciones** deben cumplirse ciertas condiciones (que surgen de su carácter de datos derivados)
 - La característica genérica que define la posibilidad de actualización es la **preservación de la clave** (reconocimiento de la tabla que 'hereda' su clave a la vista)

Creación de Vistas

```
CREATE VIEW nom_vista [(n_col_1, ..., n_col_n)]  
AS expresión_consulta  
[WITH [opción] CHECK OPTION];
```

- **nom_vista**: nombre de la vista
- **(n_col_1,..., n_col_n)**: nombres de columnas de la vista
 - Las columnas de la vista se pueden renombrar especificando la lista completa de atributos de la vista entre paréntesis.
 - Si no, los nombres son los de las columnas de las tablas especificadas en la sentencia SELECT
 - Se debe especificar con diferente nombre las columnas provenientes de distintas tablas pero con igual nombre
- **expresión_consulta**: consulta SQL que define la relación derivada

Creación de Vistas

Ejemplos

- PROV_COMP* con el identificador, nombre y ciudad de proveedores del rubro computadoras

CREATE VIEW *PROV_COMP* AS

SELECT id_proveedor, nombre, ciudad

FROM PROVEEDOR

WHERE rubro = 'Computadoras';

Vista a partir de una tabla

- PROV_COMP_TANDIL* con los proveedores de computadoras de Tandil (usando la vista previa):

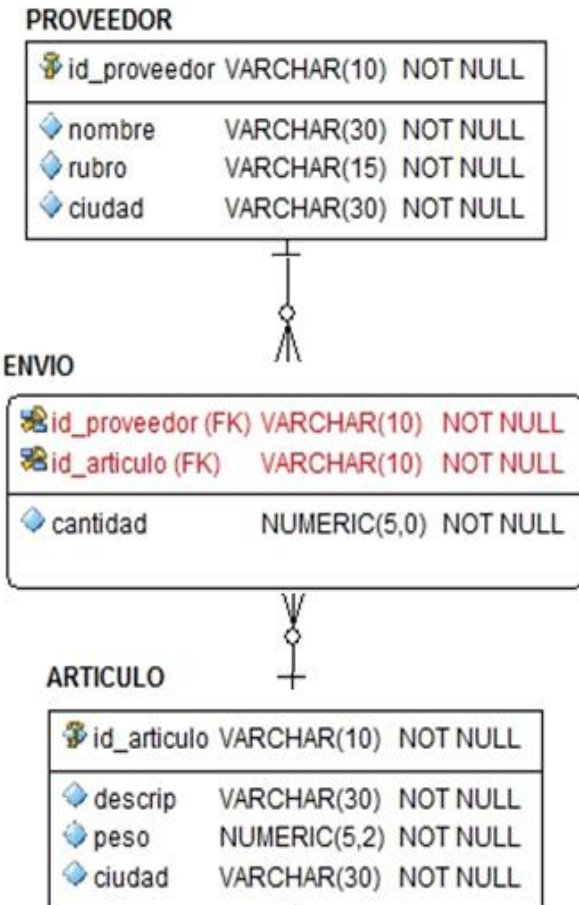
CREATE VIEW *PROV_COMP_TANDIL* AS

SELECT *

FROM *PROV_COMP*

WHERE ciudad = 'Tandil';

Vista a partir de otra vista



Creación de Vistas

Ejemplos

- *PROV_ENVIOS_TANDIL que contenga el identificador y nombre de los proveedores de computadoras de Tandil y los identificadores de artículos enviados por cada uno*

CREATE VIEW PROV_ENVIOS_TANDIL AS

```
SELECT P.id_proveedor, P.nombre, E.id_articulo  
FROM PROVEEDOR P JOIN ENVIO E  
ON P.id_proveedor = E.id_proveedor  
WHERE P.rubro = 'Computadoras'  
AND P.ciudad = 'Tandil';
```

Vista a partir de
más de una tabla

o... **CREATE VIEW PROV_ENVIOS_TANDIL AS**

```
SELECT P.id_proveedor, nombre, E.id_articulo  
FROM PR_COMP_TANDIL P JOIN ENVIO E  
ON P.id_proveedor = E.id_proveedor;
```

PROVEEDOR

id_proveedor	VARCHAR(10)	NOT NULL
nombre	VARCHAR(30)	NOT NULL
rubro	VARCHAR(15)	NOT NULL
ciudad	VARCHAR(30)	NOT NULL

ENVIO

id_proveedor (FK)	VARCHAR(10)	NOT NULL
id_articulo (FK)	VARCHAR(10)	NOT NULL
cantidad	NUMERIC(5,0)	NOT NULL

ARTICULO

id_articulo	VARCHAR(10)	NOT NULL
descrip	VARCHAR(30)	NOT NULL
peso	NUMERIC(5,2)	NOT NULL
ciudad	VARCHAR(30)	NOT NULL

Vistas – Consultas/ Eliminación

- Una vista puede ser consultada como cualquier tabla

Ejemplo

Listar alfabéticamente los proveedores de *PR_COMP_TANDIL*

```
SELECT nombre  
FROM PR_COMP_TANDIL  
ORDER BY nombre;
```

- Para eliminar una vista del esquema de la BD:

```
DROP VIEW nom_vista [opción];
```

opción=

RESTRICT → se rechaza si hay objetos que hacen referencia a la vista (*opción por defecto*)

CASCADE → procede siempre y se eliminan también los objetos dependientes

Actualizaciones Tabla Base → Vista

- Al actualizar las tuplas de una tabla → **los cambios se reflejan automáticamente sobre las vistas** definidas a partir de ella
 - Las vistas no mantienen copias de los datos (salvo que sean materializadas)
- el SGBD asegura que las vistas siempre estén actualizadas:
- por recálculo: las tuplas actualizadas se generan al acceder a la vista
 - por mantenimiento incremental (vistas materializadas)

Actualizaciones Vista → Tabla Base

- Las operaciones de actualización sobre una vista deberían propagarse automáticamente a las tablas base
- Sin embargo.. al propagarse podrían generarse ambigüedades o carecer de sentido o provocar efectos no deseados

Ejemplos

- Borrar una tupla en una vista => borrar la tupla de la tabla base?
(una modificación también podría hacerla “desaparecer” de la vista)
- Insertar una fila en una vista => insertar una tupla en la tabla base?
(si se actualiza alguna tupla también podría incorporarse en la vista)
- Cómo se propaga una actualización en caso de:
 - una vista resultante de un **ensamble**?
 - un **campo derivado**?
 - una **función de agregación**?
 - una tupla que **no conserva la clave**?
 - una selección con **distinct**?



Actualizaciones Vista → Tabla Base

- Cada actualización de una tupla en una vista debe poder **propagarse sin ambigüedades** a la/s tabla/s base o subyacente/s^(*)
- Sólo es factible cuando una actualización de la/s tabla/s base^(*) puede lograr el efecto deseado sobre la vista
- **No toda vista se puede actualizar automáticamente**
 - cuando una actualización sobre la vista pueda generar distintas posibilidades de actualización a la/s tabla/s base^(*), se deberá recurrir a algún procedimiento específico para implementar la que sea requerida

(*) Puede ser otra vista, que debe ser actualizable.

Si se tienen definidas vistas a partir de vistas : $T \rightarrow V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_n$
 V_i será actualizable si V_{i-1} lo es y así sucesivamente

Propiedad de Preservación de Clave

Establece la condición para que una vista sea actualizable:

- debe tratarse de **una única actualización y del mismo tipo** en la/s tabla/s^(*) base, cuya clave 'heredó' la vista
 - se satisface si **cada fila en la tabla aparece como máximo una vez en la vista**
 - la clave de la vista es la clave de la tabla^(*) de la cual procede (vistas de una tabla^(*))
 - la clave de la vista es la de alguna de las tablas^(*) de las cuales procede (vistas de ensamble)
- Nota: en una jerarquía de vistas, basta que una de las vistas involucradas no preserve la clave, para que esta propiedad ya no se cumpla
- La propiedad NO depende de los datos actuales en las tablas, sino que es una propiedad estructural de su esquema
 - Este concepto ha sido explicado por distintos autores (ej: Date), aplicado por el estándar SQL como forma de operar, y popularizado por Oracle que lo implementó y denominó propiedad **'key-preserved'** (los SGBD suelen hacer interpretaciones particulares)

Vistas Actualizables

(a partir de una tabla/vista)

- Según el estándar SQL, una **vista definida sobre una tabla base** (u otra vista actualizable) **es actualizable si**:
 - ✓ conserva todas las columnas de la clave (primaria)
 - ✓ no contiene funciones de agregación o información derivada
 - ✓ no incluye la cláusula DISTINCT
 - ✓ no incluye subconsultas en el SELECT

→ se denominan **vistas σ -n** (se obtienen a partir de **selección** de tuplas y **proyección** de columnas)
- El SGBD **traduce una actualización** de una vista definida a partir de una tabla/vista en una única operación de actualización del mismo tipo sobre la tabla/vista
- Esto ocurre siempre que no se viole ninguna restricción de integridad definida sobre dicha relación (ej: afectando campos que no aceptan nulos o no tienen valores por defecto definidos)
- **Nota: Algunos SGBD soportan otras posibilidades que las especificadas por el estándar SQL**

Vistas Actualizables

(a partir de una tabla/vista)

Ejemplos

- PROV_TANDIL* que contenga el identificador y nombre de los proveedores de Tandil

```
CREATE VIEW PROV_TANDIL AS  
SELECT id_proveedor, nombre  
FROM PROVEEDOR WHERE ciudad = 'Tandil';
```



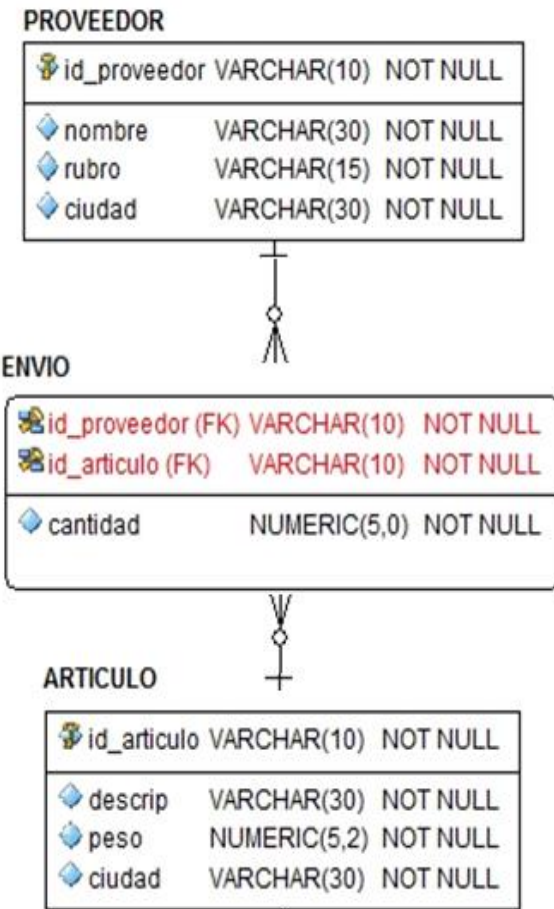
- PROV_COMP* con nombre y ciudad de proveedores de computadoras

```
CREATE VIEW PROV_COMP  
AS SELECT nombre, ciudad  
FROM PROVEEDOR WHERE rubro =  
'Computadoras';
```



- TOTAL_ARTICULO* que liste, para cada articulo, su identificador y la cantidad total enviada

```
CREATE VIEW TOTAL_ARTICULO (articulo, total) AS  
SELECT id_articulo, sum(cantidad)  
FROM ENVIO  
GROUP BY id_articulo;
```



¿Migración de Tuplas?

Al efectuar actualizaciones en una vista, podría ocurrir que las tuplas afectadas dejen de pertenecer a la vista

Ejemplo

```
CREATE VIEW PROV_TANDIL AS  
SELECT * FROM PROVEEDOR  
WHERE ciudad = 'Tandil';
```

¿Cuál sería el resultado de la siguiente operación?

```
UPDATE PROV_TANDIL SET ciudad= 'Azul';
```

- En este caso todos los registros de la vista serían actualizados con un valor de ciudad diferente a Tandil...
- y entonces dejarían de pertenecer a la vista (migración de tuplas de la vista)

Vistas con Opción de Chequeo

Solución a la migración de tuplas

→ incluir la cláusula **WITH CHECK OPTION (WCO)**

```
CREATE VIEW nom_vista [(n_col_1, ..., n_col_n)]  
AS expresión_consulta  
[WITH [opción] CHECK OPTION];
```

- Si se especifica WCO → una actualización sobre la vista procede si satisface la condición de consulta que la define (se rechaza cualquier inserción o actualización que haga *migrar* una tupla de la vista)
opción=
 - **CASCADED**: las tuplas son chequeadas contra las condiciones de la vista y aquellas de las vistas subyacentes (*opción por defecto*)
 - **LOCAL**: sólo se chequean contra las condiciones definidas en la misma vista
- Sólo está **soportado en vistas automáticamente actualizables**
- **Nota:** Date recomienda usar WCO en vistas actualizables y critica la opción 'local'

Vistas con Opción de Chequeo

Ejemplo

Vista *ENVIOS500* con los envíos de 500 o más unidades de algún artículo (a partir de ENVIO):

```
CREATE VIEW Envios500 AS  
SELECT * FROM ENVIO  
WHERE cantidad >= 500;
```

determinar el efecto de las sig. operaciones, si la vista

- se define con WCO
- se define sin WCO

- INSERT INTO *Envios500* VALUES ('P1', 'A1', 500);
- INSERT INTO *Envios500* VALUES ('P2', 'A2', 300);
- UPDATE *Envios500* SET cantidad=100 WHERE id_proveedor= 'P1';

PROVEEDOR

 id_proveedor	VARCHAR(10) NOT NULL
 nombre	VARCHAR(30) NOT NULL
 rubro	VARCHAR(15) NOT NULL
 ciudad	VARCHAR(30) NOT NULL



ENVIO

 id_proveedor (FK)	VARCHAR(10) NOT NULL
 id_articulo (FK)	VARCHAR(10) NOT NULL
 cantidad	NUMERIC(5,0) NOT NULL



ARTICULO

 id_articulo	VARCHAR(10) NOT NULL
 descrip	VARCHAR(30) NOT NULL
 peso	NUMERIC(5,2) NOT NULL
 ciudad	VARCHAR(30) NOT NULL

Vistas con Opción de Chequeo

Ejemplo

Teniendo creada *Envios500*:

```
CREATE VIEW Envios500 AS  
SELECT * FROM ENVIO  
WHERE cantidad >= 500;
```

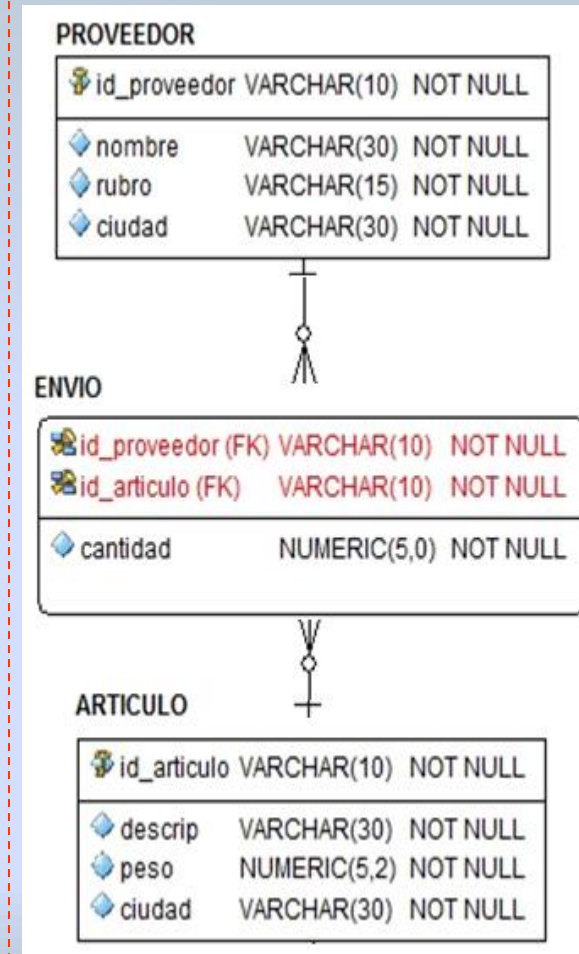
*ENVIOS500-999*_con los envíos de entre 500 y 999 unidades (a partir de *ENVIOS500*)

```
CREATE VIEW Envios500-999 AS  
SELECT * FROM ENVIO500  
WHERE cantidad < 1000;
```

Cuál es la respuesta ante la sig. operación si *Envios500-999* se define con:

- CASCADED WCO?
- LOCAL WCO?

→ UPDATE *Envios500-999* SET cantidad= 300 where (id_proveedor = 'P1' and id_articulo = 'A1');



BIBLIOGRAFÍA

Date, C., "An Introduction to Database Systems". 8º ed., Addison Wesley, 2004

Elmasri, R., Navathe, S., "Fundamentals of Database Systems", Addison Wesley, 2011 (Cap. 5)

Ramakrishnan R., Gehrke J., "Database Management Systems", 3º ed. , McGraw-Hill, 2003 (Cap. 3 y 25)

Silberschatz, A., Korth, H, Sudarshan, S., "Database System Concepts", McGraw Hill, 2001 (Cap. 4)